

GBU Grievance Assistant

An AI Chatbot for the Grievance Management & Ticket Tracking System (RAG + Groq)

Role	AI Chatbot Development & Deployment	Stack	Python / Flask / Groq / SQLite
Applicant	[Your Name]	Contact	[Your Email / Phone]

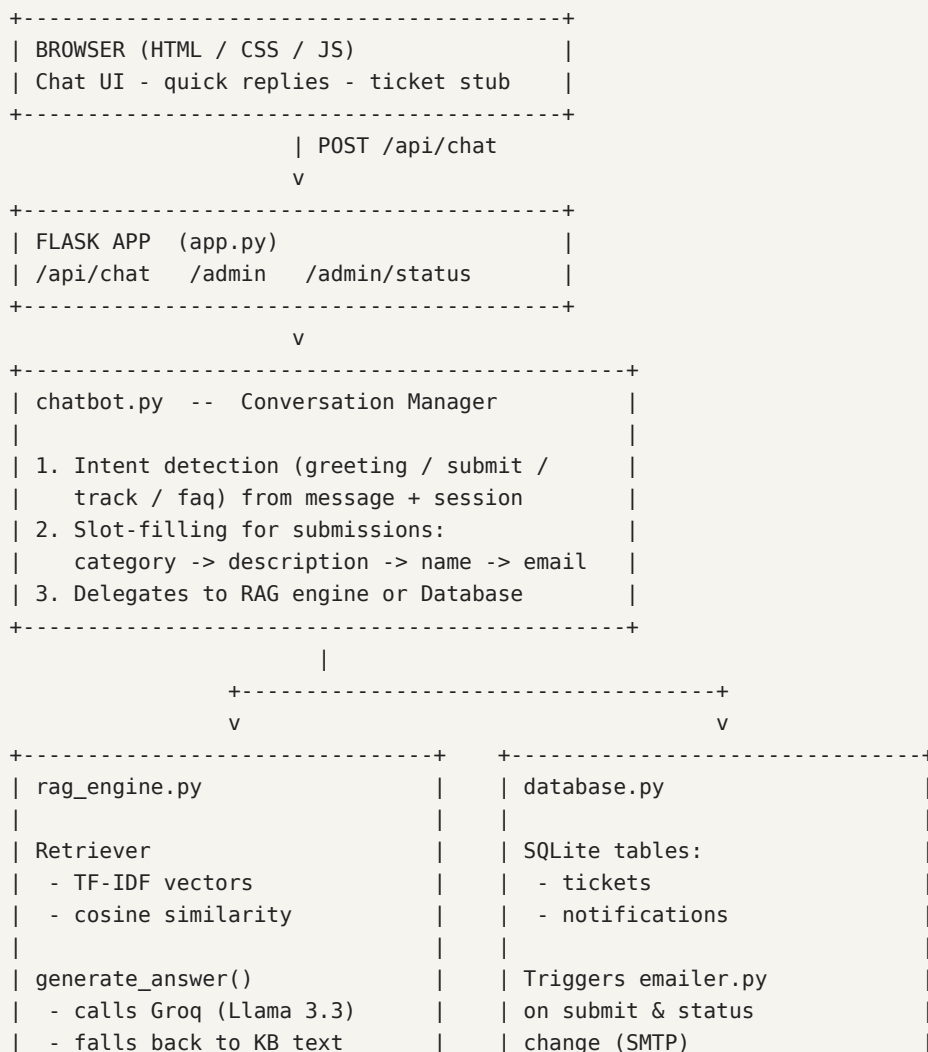
1. Objective

An AI-powered assistant embedded in the GBU Grievance Management & Ticket Tracking System that can:

- Assist students in submitting a grievance through natural conversation
- Let students track grievance status using their Ticket ID
- Answer grievance-related FAQs, grounded in an actual knowledge base (not hallucinated)
- Guide users through the overall grievance process

The design goal was a bot a real Grievance Cell could trust: every FAQ answer is traceable back to a specific knowledge-base entry, and every submit/track action is backed by a real, queryable ticket record.

2. Chatbot Architecture



```

+-----+ +-----+
|                                     |
|           v                       |
| data/knowledge_base.json          |
| (15 curated grievance-policy entries) |
|                                     |
+-----+

```

Why TF-IDF for retrieval? It runs fully offline and deterministically, needs no API key, and every retrieval score is explainable. The Retriever class is isolated so it can be swapped for sentence-embeddings later without touching the conversation manager.

Why Groq for generation? Groq's Llama 3.3 70B endpoint provides fast, low-latency inference well suited to a chat UI. The system prompt instructs the model to answer only from retrieved context, which is what keeps it grounded instead of inventing grievance policy. If GROQ_API_KEY is not set, or the call fails, the app automatically falls back to a deterministic KB-template answer -- the bot never goes down because of the LLM provider.

3. Conversation Flow

```

+-----+
| Start   |
+-----+
|
|           v
|-----+
| Detect intent from message + session state |
|-----+
|
|           |
|   +-----+ +-----+ +-----+ +-----+ +-----+
|   v         v         v         v         v
| greeting  goodbye  track_status  submit_gr.  faq

```

Each branch always loops back to 'Detect intent' for the user's next message, so a user can type 'cancel' mid-flow or ask an unrelated FAQ question mid-submission and the bot recovers gracefully.

submit_grievance branch (slot-filling):

```

+-----+
| category -> description -> name -> email |
| -> database.create_ticket() -> emailer.send_email() |
| -> reply with Ticket ID (rendered as a ticket-stub card) |
+-----+

```

track_status branch:

```

+-----+
| extract Ticket ID from message (regex) or ask for it |
| -> database.get_ticket(id) -> reply with status |
+-----+

```

faq branch (RAG):

```

+-----+
| Retriever.retrieve(message) -> top-k KB matches |
| -> score >= threshold? |
|   yes -> call_llm(Groq) if configured, else KB template |
|   no  -> "I don't know, contact IT Cell" |
+-----+

```

4. Prompt Design

This is the exact system prompt sent to Groq (rag_engine.py :: SYSTEM_PROMPT), with retrieved KB passages injected as context per request:

```
SYSTEM:
```

You are the GBU Grievance Assistant, a helpful, concise assistant for Gautam Buddha University's Grievance Management System. Answer only using the CONTEXT provided below. If the context does not contain the answer, say you don't know and suggest contacting the IT Cell / Grievance Redressal Committee. Never invent policy, timelines, or procedures that are not in the context. Keep answers under 4 sentences unless the user asks for detail. Do not repeat the user's question back to them.

CONTEXT:

- [1] (Timelines) Most grievances are acknowledged within 24-48 hours and resolved within 7 working days...
- [2] (Notifications) The system sends a mandatory email notification...

USER QUESTION:

{{ user_message }}

When no key is configured (or the call fails), `generate_answer()` falls back to the best-matching KB answer verbatim plus a related follow-up -- the same context-only, no-hallucination guarantee, just without LLM phrasing. The response payload includes a mode field ("llm" or "template") so this is visible/debuggable rather than silent.

5. Knowledge Base Structure

`data/knowledge_base.json` is an array of entries shaped like this:

```
{
  "id": "kb006",
  "category": "Timelines",
  "question": "How long does it take to resolve a grievance?",
  "answer": "Most grievances are acknowledged within 24-48 hours...",
  "keywords": ["how long", "resolution time", "turnaround", "sla"]
}
```

question and keywords are weighted 3x/2x over answer in the TF-IDF corpus, since real user queries resemble questions and keywords far more than they resemble finished answers. The current knowledge base has 15 entries:

Category	Question covered
Getting Started	How do I submit a grievance?
Getting Started	What information do I need to file a grievance?
Ticket ID	What is a Ticket ID and where do I get it?
Tracking	How can I track the status of my grievance?
Tracking	What do the grievance statuses mean?
Timelines	How long does it take to resolve a grievance?
Notifications	Will I get notified about updates to my grievance?
Categories	What categories of grievances can I file?
Escalation	What if my grievance is not resolved on time?
Editing	Can I edit or withdraw a grievance after submitting it?
Privacy	Is my grievance confidential?
Account	Do I need to log in to submit a grievance?
Support	Who do I contact for urgent issues?
Technical	The portal is not loading or showing an error, what do I do?
Fee	I was charged incorrectly, how do I raise a fee grievance?

6. Real Integrations Added

6.1 Groq LLM (generation)

Set an API key (free tier available at console.groq.com) and install the SDK:

```
export GROQ_API_KEY="your-key-here"
pip install -r requirements.txt # now includes the groq SDK
python app.py
```

6.2 SMTP email notifications

The "Grievance Submitted" and "Status Updated" emails are sent for real via `emailer.py` using `smtplib`, configured entirely through environment variables:

```
export SMTP_HOST="smtp.gmail.com"
export SMTP_PORT="587"
export SMTP_USER="your.address@gmail.com"
export SMTP_PASSWORD="16-char app password"
```

For Gmail, generate an App Password (Google Account -> Security -> 2-Step Verification -> App Passwords) rather than using the normal account password. If these variables aren't set, every notification is still recorded in the notifications table (`delivered=0`) so the requirement stays visibly wired end-to-end, and the app remains fully runnable with zero setup for a quick demo.

7. Project Structure

```
gbu-chatbot/
├── app.py                Flask routes (chat API, admin demo view)
├── chatbot.py            Conversation manager: intents, slot-filling
├── rag_engine.py         Retriever (TF-IDF) + Groq LLM call
├── emailer.py            Real SMTP email sending
├── database.py           SQLite: tickets + notifications
├── data/
│   └── knowledge_base.json  RAG knowledge base (15 entries)
├── templates/
│   ├── index.html        Chat UI
│   └── admin.html         Demo admin view
├── static/
│   ├── style.css          GBU-branded styling
│   ├── script.js          Chat interactivity
│   └── requirements.txt
└──
```

8. Running the Prototype

```
pip install -r requirements.txt
python app.py
```

Then open `http://127.0.0.1:5000` for the chat assistant, and `http://127.0.0.1:5000/admin` to see submitted tickets and simulate an admin updating status.

9. Evaluation Criteria Self-Check

Criteria	How this prototype addresses it
Problem Understanding	Bot covers all 4 required capabilities: submit, track, FAQ, guidance
Creativity & Innovation	Ticket-stub UI motif, RAG source-citation chips, graceful mid-flow interruption, generation-mode badge (Groq vs. KB fallback)
Technical Approach	Real RAG pipeline (TF-IDF retrieval + Groq LLM generation), DB-backed tickets, real SMTP notifications, clean separation of concerns
Documentation Quality	This document: architecture, flow, prompt design, KB structure
Practical Feasibility	Runs with zero external dependencies; Groq + SMTP are opt-in via env vars with automatic graceful fallback

10. What Would Change for Production

- Swap the Retriever for sentence embeddings + a vector index (FAISS/pgvector) once the KB grows beyond a few hundred entries
- Replace the in-memory session dict with Redis so the bot works across multiple server workers
- Move SMTP credentials and `GROQ_API_KEY` into a proper secrets manager

- Add authentication so session_id maps to a verified GBU student identity
- Add retry/backoff around the Groq call and basic per-session rate limiting